

PROJECT REPORT – AI-BASED URBAN TRAFFIC CONGESTION PREDICTION SYSTEM

AI-Based Urban Traffic Congestion Prediction System

A machine-learning pipeline that predicts per-road-segment congestion ahead of time and visualises it on a live dashboard.

Collect → Predict → Visualise

Stages 1–8 complete

450 automated tests passing

Report + deck included

Problem Statement

- Congestion is discovered too late – commuters learn about it when they are already inside it.
- Reactive traffic control answers “where is it congested now?”, not “will this road be congested in 30 minutes?”.
- Static timetables cannot capture incidents, weather and peak-hour variation.
- The question that changes behaviour is forward-looking and per road segment.
- The gaps a data-driven system must fill:
 - no vendor publishes traffic throughput, so vehicle_count is stored as unknown
 - any system that pretends to know real traffic volume is fabricating it

Need: a forward-looking per-segment congestion estimate that is honest about what it can and cannot know.

Objectives and Approach

- Collect – real urban traffic observations into PostgreSQL, with provenance on every row.
- Engineer – leakage-safe features from time, road geometry and weather.
- Train – a Random Forest classifier and score it against an honest majority-class baseline.
- Serve – documented REST API returning one error envelope.
- Show – current and predicted congestion on a map and charts.
- Guard – boot-time production guard so a misconfigured deployment fails loudly instead of silently serving simulation.

Every result traceable to data; nothing fabricated.

Proposed Solution

Data ingestion

- provider interface
- labelled simulation OR live TomTom + Open-Meteo

Pipeline + model

- 6-stage, leakage-controlled
- Random Forest v4.0.0, 26 features
- artifacts carry provenance

Prediction engine

- scored against segment history
- insufficient history = explicit error

Dashboard API + frontend

- 5 aggregation endpoints
- Leaflet map + Chart.js analytics

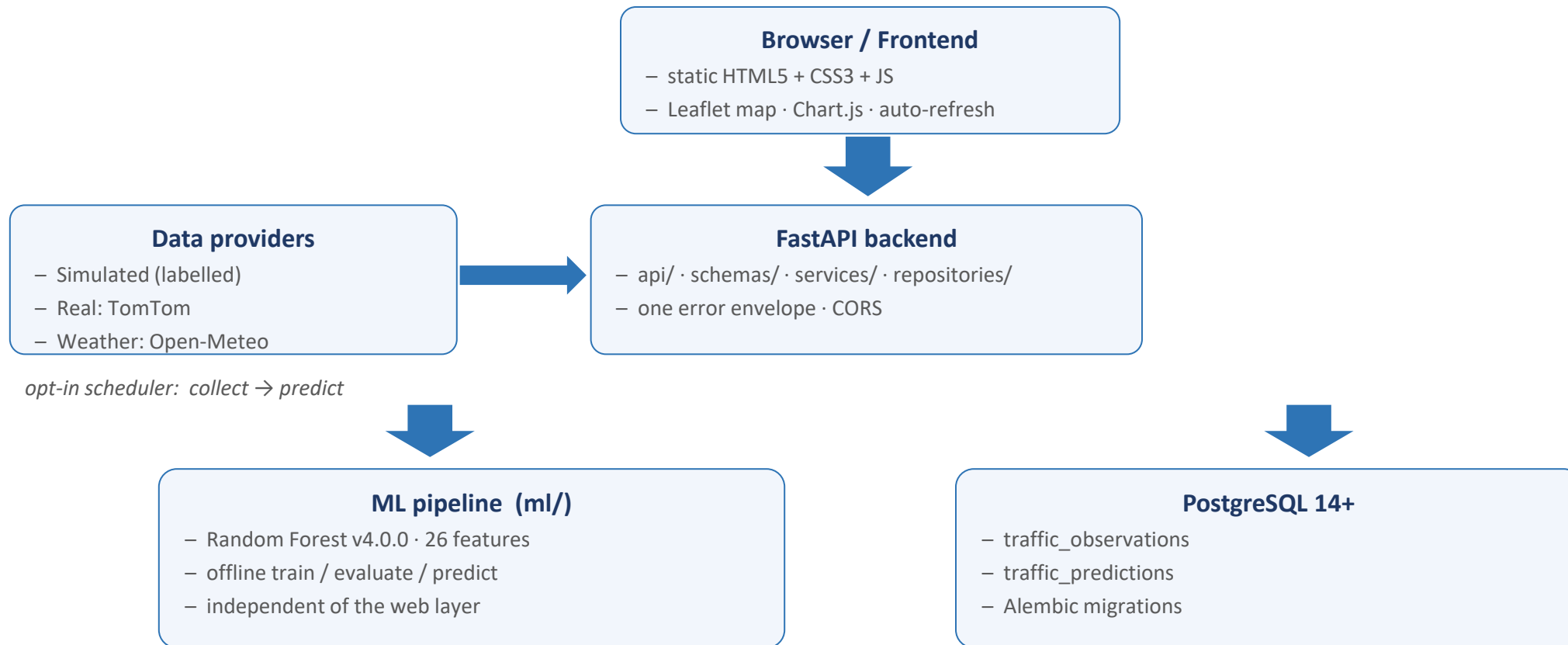
Production guard

- no silent simulation
- fails at boot naming the missing setting

One interface, two providers, one honest pipeline

Nothing downstream builds on an unproven assumption.

System Architecture



Data & ML Pipeline

Six-stage offline pipeline



- Leakage controls: contemporaneous measurements refused at training AND at the API boundary; backward-only lags; chronological split.
- Random Forest – strong on tabular data, little tuning, feature importances explain predictions.
- 4 congestion levels (free flow / moderate / heavy / severe).
- Macro F1 is the headline metric – accuracy optimisers ignore rare severe congestion.
- Every artifact stores its own feature order + provenance – no train/serve feature-skew failure mode, and the fixture is labelled simulated.

Real-Time Traffic Collection

Provider interface

- SimulatedTrafficProvider – labelled, always
- RealTrafficProvider – TomTom Flow Segment Data
- WeatherProvider – Open-Meteo
- one code path for scheduled + manual POST /collect

Collection behaviour

- throttled per cycle
- opt-in via COLLECTION_ENABLED=true
- timestamped with source provenance on every row
- stored first, predicted second (auto-prediction)

Honesty + failure mode

- missing provider key fails loudly, never silently simulated
- vehicle_count unknown → stored as NULL (no vendor publishes throughput)
- production requires TRAFFIC_PROVIDER=real + TRAFFIC_API_KEY
- no self-serve throughput API – a system claiming volume fabricates it

Live outside the suite: Open-Meteo call verified (HTTP 200). TomTom needs a real key.

Weather Integration

Open-Meteo – no API key required



appended to each observation before featurisation

- Live call verified in this project: HTTP 200, fields `current.temperature_2m`, `current.precipitation`, `current.weather_code`.
- Weather cycles share the throttled provider pipeline – one cycle, weather + traffic together.
- Weather fields become model features alongside time and road geometry.
- In production the weather provider keeps working with `TRAFFIC_PROVIDER=real`.
- Weather is standard practice for congestion models – rain and temperature shift speed distributions.

No key, no fabrications – the weather source was exercised live.

Dashboard

- Leaflet map coloured by predicted congestion class (free / moderate / heavy / severe).
- Chart.js trend chart – speed vs forecast over 1 / 6 / 12 / 24 hours.
- Auto-refresh; live traffic tab → map → analytics → system status.
- Five aggregation endpoints: current, trends, locations, status, model – fixed queries, no N+1.
- Honest empty states – “N/A” is never a fabricated zero; uncollected roads report null and unavailable.
- Model provenance shown next to every forecast (dataset_is_simulated: true), and every API-derived string is HTML-escaped before it reaches the page.

Testing & Results

Verification – 450 automated tests • observe → predict → persist
against the real artifact

- every dashboard endpoint, freshness boundaries, fixed query counts
- Alembic: apply → verify → reverse → re-apply + offline PostgreSQL DDL
- provider contracts via mocked transports; security + secret-leak tests
- Stage 8: production-guard + XSS tests
- no external services required to run the suite

On the simulated fixture

macro F1 0.737

accuracy 0.856

vs majority-class baseline

macro F1 0.214 • accuracy 0.750

These numbers show the pipeline runs – they are not a statement about real traffic.

Benefits · Limitations · Future

Benefits

- honest, fully labelled pipeline
- leakage-safe evaluation with baseline
- retraining-ready for a real dataset
- provenance removes feature skew
- production guard + XSS hardening
- no external services needed to test

Limitations

- training data is a simulated fixture
- vehicle_count NULL on live readings
- no historical feed yet – scheduler accumulates it
- API unauthenticated until a gateway is added

Future

- retrain + revalidate on real ground truth
- Docker image + CI running the suite
- gateway auth and rate limiting
- live TomTom + PostgreSQL round-trip

A working, honest end-to-end pipeline

- Collect → predict → visualise, verified by 450 automated tests.
- Leakage-safe machine learning with provenance on every artifact.
- Demonstrated – not field-validated: forecasts improve once real ground truth flows through the same tested pipeline.
- Ready for retraining, containerisation and public deployment hardening.

Questions.

Project report, deployment guide and full API docs shipped alongside the deck.